Lista projektów 13.03.2026 r. termin oddania projektu 27.03.2026 r:

Polecenie do każdego projektu

1. **Utworzyć bazę danych** wraz z odpowiednimi opcjami konfiguracyjnymi.
2. **Utworzyć tabele** w bazie danych, definiując odpowiednie pola, typy danych oraz ograniczenia (np. klucze główne, klucze obce).
3. **Dodać przykładowe dane** do tabel przy użyciu instrukcji `INSERT`.
4. **Przygotować zapytania SQL typu `SELECT`**, w których należy zastosować:
   a. funkcje agregujące, np. `SUM()`, `COUNT()`, `AVG()` itp.,
   b. klauzulę `GROUP BY`,
   c. klauzulę `HAVING`,
   d. sortowanie wyników przy użyciu `ORDER BY`.
5. **W projekcie muszą znaleźć się minimum jedno zapytania `SELECT`** spełniające powyższe wymagania.

Sposób oddania projektu

Projekt należy **wysłać na mój adres e-mail podany na lekcji**.
Tytuł wiadomości e-mail powinien mieć format:
1A gr 1 – Imię Nazwisko
W treści wiadomości należy wpisać:

- temat projektu.

Załącznik do wiadomości:

- plik z projektem należy **spakować do archiwum o rozszerzeniu .7z** (np. przy użyciu programu **7-Zip**),
- archiwum powinno zawierać wszystkie pliki związane z projektem.
  Przykład nazwy pliku:
  Jan_Kowalski_Systemy_baz_danych.7z

1. Sklep internetowy

Bazy: ecommerce
Tabele: users, products, orders
Wymagania:

- orders.user_id → FK users.id z ON DELETE RESTRICT
- products.price → CHECK (price > 0)
- AUTO_INCREMENT dla id w każdej tabeli
- **Agregacja:** liczba zamówień na użytkownika, HAVING COUNT(orders.id) > 5

2. Biblioteka

Bazy: library_db
Tabele: books, authors, borrowers, loans
Wymagania:

- loans.book_id → FK books.id z ON DELETE CASCADE
- loans.borrow_date → NOT NULL
- **Agregacja:** ile książek wypożyczył każdy czytelnik, HAVING COUNT(loans.id) > 3

3. Szkoła / uczniowie

Bazy: school_db
Tabele: students, teachers, classes, grades
Wymagania:

- grades.student_id → FK students.id
- grades.value → CHECK (value BETWEEN 2 AND 5)
- **Agregacja:** średnia ocen uczniów w klasach, HAVING AVG(grades.value) > 4.0

4. Firma / pracownicy

Bazy: company_db
Tabele: employees, departments, projects, assignments
Wymagania:

- assignments.emp_id → FK employees.id
- salary → CHECK (salary > 0)
- **Agregacja:** liczba projektów na pracownika, HAVING COUNT(assignments.project_id) > 2

5. Szpital / pacjenci

Bazy: hospital_db
Tabele: patients, doctors, appointments
Wymagania:

- appointments.patient_id → FK patients.id
- appointments.date → NOT NULL
- **Agregacja:** ilu pacjentów ma każdy lekarz, HAVING COUNT(appointments.id) > 10

6. Kino / repertuar

Bazy: cinema_db
Tabele: movies, halls, sessions, tickets
Wymagania:

- tickets.session_id → FK sessions.id
- tickets.price → CHECK (price >= 0)
- **Agregacja:** liczba biletów sprzedanych na film, HAVING SUM(tickets.price) > 1000

7. Restauracja

Bazy: restaurant_db
Tabele: menu_items, orders, order_items, waiters
Wymagania:

- order_items.menu_item_id → FK menu_items.id
- order_items.quantity → CHECK (quantity > 0)
- **Agregacja:** suma zamówionych porcji dla każdego dania, HAVING SUM(order_items.quantity) > 20

8. Forum / użytkownicy

Bazy: forum_db
Tabele: users, posts, comments
Wymagania:

- FK comments.post_id → posts.id
- posts.title → NOT NULL
- **Agregacja:** liczba postów na użytkownika, HAVING COUNT(posts.id) > 10

9. Muzeum -

Bazy: museum_db
Tabele: artworks, artists, exhibitions, artwork_exhibition
Wymagania:

- FK artwork_exhibition.artwork_id → artworks.id
- artworks.year → CHECK (year > 0)
- **Agregacja:** ile dzieł ma każdy artysta, HAVING COUNT(artworks.id) > 5

10. Siłownia / klienci

Bazy: gym_db
Tabele: members, trainers, sessions
Wymagania:

- FK sessions.trainer_id → trainers.id
- sessions.duration → CHECK (duration > 0)
- **Agregacja:** liczba treningów dla trenera, HAVING COUNT(sessions.id) > 15

11. Sklep muzyczny

Bazy: music_store_db
Tabele: albums, artists, sales
Wymagania:

- FK sales.album_id → albums.id
- sales.quantity → CHECK (quantity > 0)
- **Agregacja:** liczba sprzedanych egzemplarzy dla każdego albumu, HAVING SUM(sales.quantity) > 50

12. Hotel / rezerwacje

Bazy: hotel_db
Tabele: guests, rooms, bookings
Wymagania:

- FK bookings.guest_id → guests.id
- bookings.check_in i check_out → NOT NULL
- **Agregacja:** liczba rezerwacji dla każdego gościa, HAVING COUNT(bookings.id) > 3

13. Lotnisko / loty

Bazy: airport_db
Tabele: flights, airplanes, crew_members, flight_assignments
Wymagania:

- FK flight_assignments.flight_id → flights.id
- FK flight_assignments.crew_id → crew_members.id
- **Agregacja:** liczba lotów przypisanych do załogi, HAVING COUNT(flight_assignments.flight_id) > 5

14. Sklep spożywczy

Bazy: grocery_db
Tabele: products, categories, purchases, purchase_items
Wymagania:

- FK purchase_items.product_id → products.id
- purchase_items.quantity → CHECK (quantity > 0)
- **Agregacja:** ile sztuk każdego produktu zostało kupionych, HAVING SUM(purchase_items.quantity) > 10

15. Serwis samochodowy

Bazy: car_service_db
Tabele: cars, customers, repairs, repair_items
Wymagania:

- FK repairs.car_id → cars.id
- FK repair_items.repair_id → repairs.id
- **Agregacja:** liczba napraw dla klienta, HAVING COUNT(repairs.id) > 2

16. Portal ogłoszeniowy

Bazy: ads_db
Tabele: users, ads, categories, ad_views
Wymagania:

- FK ads.user_id → users.id
- FK ad_views.ad_id → ads.id
- **Agregacja:** liczba wyświetleń dla każdego ogłoszenia, HAVING COUNT(ad_views.id) > 50